Práctica 6 de Lógica Computacional

Diferencias de listas

De los programas conocidos a las listas abiertas

Erik Figueiral Alonso

Apuntes: sección 11.2.1, páginas 172–178.

Índice

1 **Representación y concatenación**

2 Añadir al final: inverso

3 Añadir al principio y concatenar: aplanar y cartesiano

4 Añadir en medio: quicksort

5 Ejercicios: recorridos de árboles

El orden de los argumentos cambia el coste

```
concat([],L,L).
concat([C|Cs],L,[C|R]) :- concat(Cs,L,R).
```

L contiene un millón de unos. El resultado es el mismo.

Consulta	Trabajo de concat/3
concat(L,[],R)	Recorre y reconstruye un millón de celdas.
concat([],L,R)	Unifica R con L en el caso base.

El coste depende de la longitud del primer argumento.

La concatenación clásica recorre la primera lista

Llamada	Resultado que queda por construir
concat([1,1,1],[],R)	R = [1|R1]
concat([1,1],[],R1)	R1 = [1|R2]
concat([1],[],R2)	R2 = [1|R3]
concat([],[],R3)	R3 = []

Buscamos eliminar este recorrido para enlazar dos resultados.

Una lista delimitada por dos referencias

```
X \ Y
```




```
X = [1,2,3,4,5]        Y = [3,4,5]
```

X\Y representa [1,2]: desde X hasta antes de la cola Y.

Sección 11.2.1, página 172. Las referencias ilustran términos y colas compartidas.

Varias representaciones de la misma lista

Diferencia de listas	Segmento representado
[1,2] \ []	[1,2]
[1,2,3] \ [3]	[1,2]
[1,2,3,4] \ [3,4]	[1,2]
[1,2|T] \ T	[1,2]

Con T libre, la última representación deja abierta la cola.

El operador aporta notación

```
:- op(600,xfy,[\]).
```

Usaremos esta directiva. Veremos los operadores en la próxima clase.
Aquí declara prioridad 600 y asociación por la derecha.

Notación	Interpretación en estos programas
X\Y X-Y X+Y	Un término con dos argumentos.
Unificación	Relaciona los extremos de cada segmento.

No hay resta de conjuntos ni evaluación aritmética automática.

Concatenar: compartir el extremo intermedio

```
concatenar_dl(X\Y, Y\Z, X\Z).
```



La cola de la primera y el comienzo de la segunda son la misma Y.

Diagrama adaptado de la figura 11.1, página 173.

Un ejemplo con X, Y y Z concretos

```
X = [1,2,3,4,5]
Y = [3,4,5]
Z = []
```

X\Y = [1,2]



Y\Z = [3,4,5]



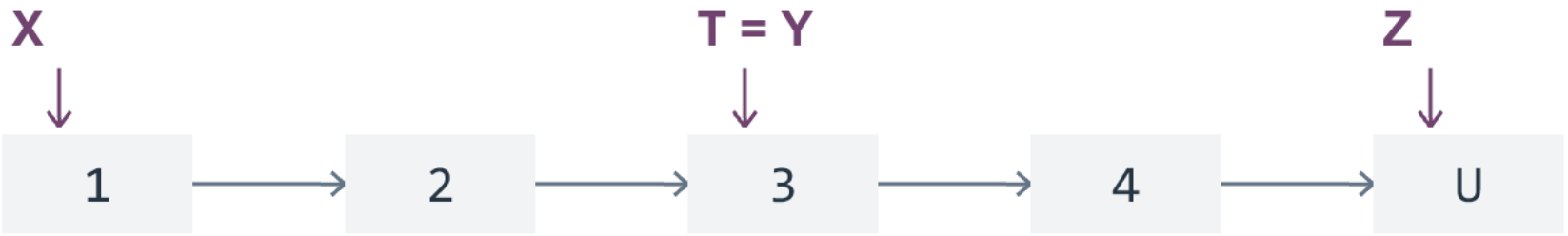
X\Z = [1,2,3,4,5]



Los dos segmentos comparten la misma lista subyacente.

Enlazar dos listas abiertas

```
?- concatenar_dl([1,2|T]\T, [3,4|U]\U, D).
T = [3,4|U],
D = [1,2,3,4|U]\U.
```



Con una cola libre y sin ciclos, el enlace cuesta O(1).

La unificación enlaza términos. Comparar colas ya instanciadas puede requerir recorrido.

La lista vacía es X con la misma X

```
X\X
```

Representación	¿Representa el segmento vacío?
[]\[]	Sí.
[1]\[1]	Sí: comienzo y final coinciden.
T\T	Sí: conserva la cola que necesitemos.

El caso base debe aceptar cualquier cola: X\X, no solo []\[].

Una interfaz entrega una lista clásica

```
inverso(Lista,Inv) :- inv_dl(Lista,Inv\[]).
```

El usuario llama a inverso/2.
El programa interno trabaja con diferencias de listas.
La interfaz fija la cola final a [].

```
?- inverso([1,2,3],R).
R = [3,2,1].
```

Inverso: añadir Car al final

```
inv([],[]).
inv([Car|Cdr],R) :-
    inv(Cdr,T), concat(T,[Car],R).
```

Para invertir [1,2,3], invertimos [2,3] y añadimos 1 al final.

Parte recursiva	Elemento final	Resultado
[3,2]	1	[3,2,1]

Con diferencias de listas evitamos recorrer T para añadir Car.

Inverso con diferencias de listas

```
inverso(Lista,Inv) :- inv_dl(Lista,Inv\[]).

inv_dl([],X\X).
inv_dl([Car|Cdr],X\Y) :-
    inv_dl(Cdr,X\[Car|Y]).
```

La llamada recursiva deja como cola [Car|Y].
El segmento exterior incluye ese Car y termina en Y.

Se mantiene la recursión sobre la entrada. Desaparece concat/3.

Inverso: seguir los argumentos

Entrada pendiente	Diferencia pedida
[1,2,3]	R\[]
[2,3]	R\[1]
[3]	R\[2,1]
[]	R\[3,2,1]

El caso X\X unifica R con [3,2,1].

Dónde termina cada segmento del inverso



Inverso de [2,3,4,5]



Inverso de [1,2,3,4,5]



La llamada interior termina antes de 1. La exterior lo incluye.

La respuesta interna conserva la cola abierta

```
?- inv_dl([1,2,3],D).
D = [3,2,1|T]\T.
```

```
?- inverso([1,2,3],R).
R = [3,2,1].
```

T es una variable libre. SWI-Prolog puede mostrarla como _A o _1234.
La interfaz cierra la cola y entrega la lista que espera el usuario.

Ejercicio: traducir aplanar

```
apla(Lista,Resultado).
```

Resultado contiene las hojas de Lista, en el mismo orden.
Las listas vacías no aportan elementos.

```
?- apla([1,[a,[],b],2],R).
R = [1,a,b,2].
```

Implementa la interfaz y aplanar_dl/2 con el operador -.

Solución: aplanar enlazando dos segmentos

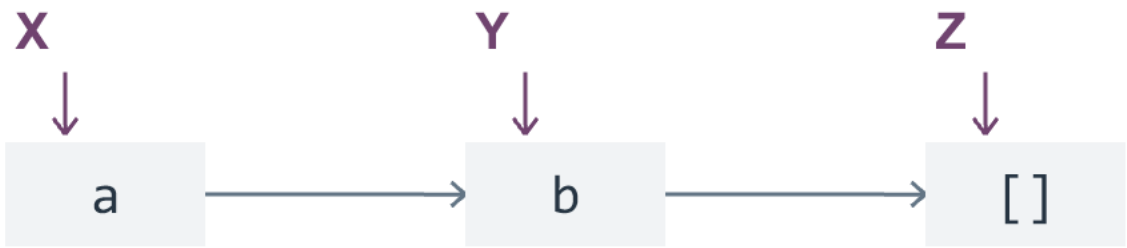
```
apla(L,R) :- aplanar_dl(L,R-[]).

aplanar_dl([],X-X).
aplanar_dl(A,[A|T]-T) :- atomic(A), A \== [].
aplanar_dl([Car|Cdr],X-Z) :-
    aplanar_dl(Car,X-Y),
    aplanar_dl(Cdr,Y-Z).
```

Un átomo o número aporta una celda al principio.
La lista compuesta enlaza el resultado de Car con el de Cdr.

Aplanar [a,[b]]: la cola común

Objetivo	Relación entre los extremos
aplanar_dl(a,X-Y)	X = [a|Y]
aplanar_dl([[b]],Y-[])	Y = [b]
Resultado	X = [a,b]



Ejemplo 11.2.2, páginas 175–176. Se omiten las llamadas que solo aportan segmentos vacíos.

Ejercicio: construir una línea del cartesiano

```
linea_dl(Elemento,Lista,Diferencia).
```

Cada elemento de Lista forma un par con Elemento.
Los conjuntos de entrada no tienen elementos repetidos.

```
?- linea_dl(1,[a,b],R-[]).
R = [[1,a],[1,b]].
```

El auxiliar trabaja directamente con diferencias de listas.

Solución: una celda al principio de la línea

```
linea_dl(_,[],X-X).
linea_dl(E,[Car|Cdr],[[E,Car]|X]-Y) :-
    linea_dl(E,Cdr,X-Y).
```

Llamada	Resultado
linea_dl(1,[b],X-[])	X = [[1,b]]
linea_dl(1,[a,b],R-[])	R = [[1,a]|X]

El resultado mantiene el orden: [[1,a],[1,b]].

La variante que añade los pares al final

```
linea_final_dl(_,[],X-X).
linea_final_dl(E,[Car|Cdr],X-Y) :-
    linea_final_dl(E,Cdr,X-[[E,Car]|Y]).
```

Versión con [a,b]	Resultado al cerrar la cola
Añadir al principio	[[1,a],[1,b]]
Añadir al final tras la recursión	[[1,b],[1,a]]

Son los mismos pares como conjunto. Cambia el orden de la lista.

Cartesiano: concatenar las líneas

```
cart(C1,C2,R) :- cart_dl(C1,C2,R-[]).

cart_dl([],_,X-X).
cart_dl([Car|Cdr],C,X-Z) :-
    linea_dl(Car,C,X-Y),
    cart_dl(Cdr,C,Y-Z).
```

```
?- cart([1,2],[a,b],R).
R = [[1,a],[1,b],[2,a],[2,b]].
```

¿Podemos llamar a la linea/3 clásica?

```
cart_dl([Car|Cdr],C,X-Z) :-
    linea(Car,C,X-Y),
    cart_dl(Cdr,C,Y-Z).
```

linea/3 devuelve una lista clásica.
¿Qué ocurrirá al pedirle un término X-Y como resultado?

Compruébalo con Car = 1 y C = [a,b].

Cada auxiliar debe devolver la representación esperada

[[1,a],[1,b]] tiene estructura de lista.
X-Y tiene como símbolo principal - y dos argumentos.
No unifican: necesitamos linea_dl/3.

Primera línea: X-Y



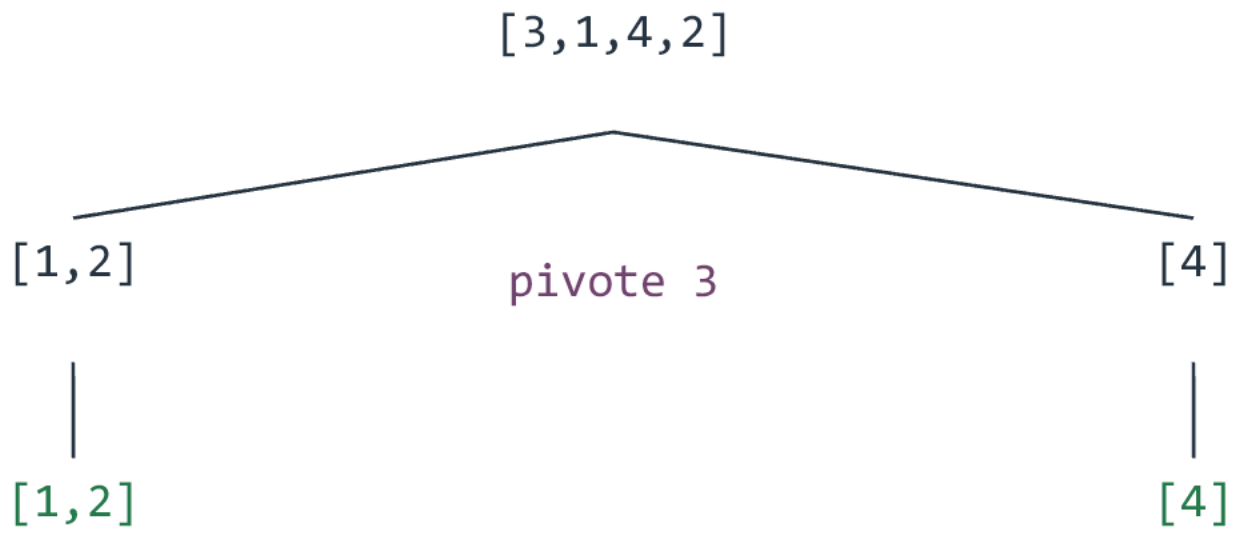
Resto: Y-Z



Cartesiano: X-Z



Quicksort: el pivote queda en medio



Ordenar la izquierda + colocar 3 + ordenar la derecha

```
Resultado = [1,2,3,4]
```

Partir sigue trabajando con listas clásicas

```
partir(_,[],[],[]).
partir(P,[C|Cs],[C|Izq],Der) :-
    C =< P, !, partir(P,Cs,Izq,Der).
partir(P,[C|Cs],Izq,[C|Der]) :-
    partir(P,Cs,Izq,Der).
```

```
?- partir(3,[1,4,2,3],Izq,Der).
Izq = [1,2,3], Der = [4].
```

Izq: menores o iguales al pivote. Der: mayores.

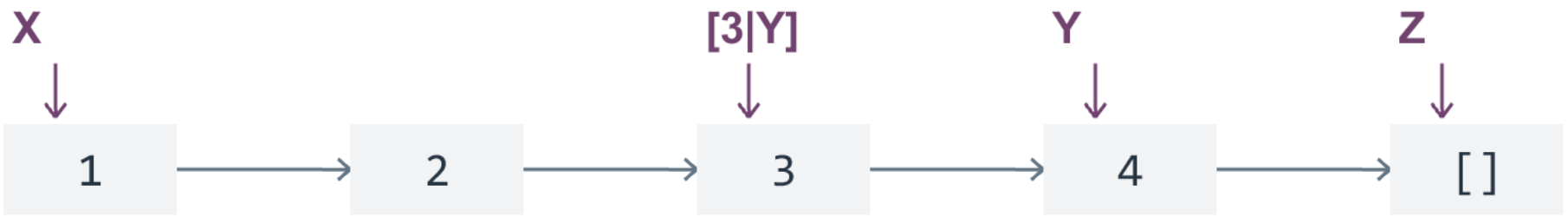
Quicksort con diferencias de listas

```
quick(L,R) :- quicksort_dl(L,R+[]).

quicksort_dl([],X+X).
quicksort_dl([Car|Cdr],X+Z) :-
    partir(Car,Cdr,Izq,Der),
    quicksort_dl(Izq,X+[Car|Y]),
    quicksort_dl(Der,Y+Z).
```

El + construye un término. Aquí no estamos sumando números.

Los extremos colocan el pivote entre ambas partes



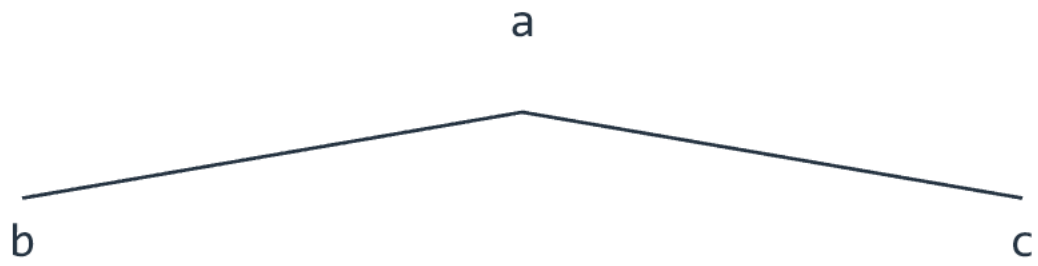
Izquierda

Derecha

X+[3|Y] acaba antes de 3. Y+Z empieza después de 3.
X+Z incluye izquierda, pivote y derecha: [1,2,3,4].

Los árboles de los ejercicios

```
nil
arbol(Raiz,Izquierdo,Derecho)
```




```
arbol(a,arbol(b,nil,nil),arbol(c,nil,nil))
```

Tres órdenes para visitar el mismo árbol

Recorrido	Orden de visita	Ejemplo anterior
Preorden	Raíz, izquierdo, derecho	[a,b,c]
Inorden	Izquierdo, raíz, derecho	[b,a,c]
Posorden	Izquierdo, derecho, raíz	[b,c,a]

Primero implementad la versión clásica y probadla. Después, traducidla.

Ejercicio de examen: inorden/2

```
inorden(Arbol,Lista).
```

Lista contiene las etiquetas en inorden.
Implementa la versión clásica y después una versión sin append/3.

Árbol	Resultado esperado
arbol(a,nil,nil)	[a]
arbol(a,arbol(b,nil,nil),nil)	[b,a]
arbol(a,nil,arbol(b,nil,nil))	[a,b]

Adaptado de DifListas-16-17: LC1–LC2. Enunciados disponibles en Moovi.

Ejercicio de examen: preorden/2

```
preorden(Arbol,Lista).
```

Lista contiene las etiquetas en preorden.
Prueba primero la versión clásica y traduce después a diferencias.

Árbol	Resultado esperado
arbol(a,nil,nil)	[a]
arbol(a,arbol(b,nil,nil),nil)	[a,b]
arbol(a,nil,arbol(b,nil,nil))	[a,b]

Adaptado de DifListas-16-17: LC3–LC4. La versión traducida no utiliza append/3.

Ejercicio de examen: posorden/2

```
posorden(Arbol,Lista).
```

Lista contiene las etiquetas en posorden o postorden.
Entrega una interfaz que devuelva una lista clásica.

Árbol	Resultado esperado
arbol(a,nil,nil)	[a]
arbol(a,arbol(b,nil,nil),nil)	[b,a]
arbol(a,nil,arbol(b,nil,nil))	[b,a]

Adaptado de DifListas-16-17: LC5. Primero versión clásica, después diferencias de listas.

Solución clásica: inorden

```
inorden_c(nil,[]).
inorden_c(arbol(A,I,D),R) :-
    inorden_c(I,RI),
    inorden_c(D,RD),
    append(RI,[A|RD],R).
```

Tratamos nil como árbol vacío. Así basta un único caso recursivo.
La raíz A queda entre los recorridos izquierdo y derecho.

Versión equivalente simplificada: incorpora explícitamente el árbol vacío.

Soluciones clásicas: preorden y posorden

```
preorden_c(nil,[]).
preorden_c(arbol(A,I,D),[A|R]) :-
    preorden_c(I,RI), preorden_c(D,RD),
    append(RI,RD,R).
```

```
posorden_c(nil,[]).
posorden_c(arbol(A,I,D),R) :-
    posorden_c(I,RI), posorden_c(D,RD),
    append(RI,RD,T), append(T,[A],R).
```

Solución: inorden con la raíz en medio

```
inorden(A,R) :- inorden_dl(A,R-[]).

inorden_dl(nil,X-X).
inorden_dl(arbol(A,I,D),X-Z) :-
    inorden_dl(I,X-[A|Y]),
    inorden_dl(D,Y-Z).
```



Solución: preorden con la raíz al principio

```
preorden(A,R) :- preorden_dl(A,R-[]).

preorden_dl(nil,X-X).
preorden_dl(arbol(A,I,D),[A|X]-Z) :-
    preorden_dl(I,X-Y),
    preorden_dl(D,Y-Z).
```

[A|X] coloca primero la raíz.
X-Y aporta el hijo izquierdo. Y-Z aporta el derecho.

Solución: posorden con la raíz al final

```
posorden(A,R) :- posorden_dl(A,R-[]).

posorden_dl(nil,X-X).
posorden_dl(arbol(A,I,D),X-Z) :-
    posorden_dl(I,X-Y),
    posorden_dl(D,Y-[A|Z]).
```

X-Y aporta el hijo izquierdo.
Y-[A|Z] aporta el derecho y deja A para el final.